

一、实验目的

了解 LR(0)语法分析算法的基本思想，掌握 LR(0)语法分析程序的构造方法。

二、实验内容

根据 LR(0)语法分析算法的基本思想，设计一个对给定文法进行 LR(0)语法分析的程序，并用 C、C++或 Java 语言编程实现。要求程序能够对从键盘输入的任意字符串进行分析处理，判断出该输入串是否是给定文法的有效句子，并针对该串给出具体的 LR(0)语法分析过程。

三、实验要求

1、已知文法 $G[S']$:

(0) $S' \rightarrow E$

(1) $E \rightarrow aA$

(2) $E \rightarrow bB$

(3) $A \rightarrow cA$

(4) $A \rightarrow d$

(5) $B \rightarrow cB$

(6) $B \rightarrow d$

建立文法 $G[S']$ 的 LR(0)分析表。

2、根据编译原理教材上算法思想，构造 LR(0)语法分析程序。

3、用该 LR(0)语法分析程序对任意给定的键盘输入串 `bccd#`进行语法分析，并根据栈的变化状态输出给定串的具体分析过程。

四、运行结果示例

从任意给定的键盘输入串：

`bccd#`

输出：

用 LR(0)分析法分析符号串 `bccd#`的过程

五、实验提示

本实验重点有两个：一是如何用适当的数据结构实现 LR(0)分析表的存储和使用；二是如何实现遇到 `rj` 值时的归约处理。

建议：使用结构体数组。

六、分析与讨论

1、若输入串不是指定文法的句子，会出现什么情况？

2、总结 LR(0)语法分析程序的设计和实现的一般方法。

个人实验代码（分析步骤略）：

```
#include<iostream>
#include<cstring>
#include<cstdio>
#include<algorithm>
#include<stack>
using namespace std;

//表格数组
          a          b          c          d          #          E          A
B
char LR0[50][50][100] = {{ "S2"      , "S3"      , "null" , "null" , "null" , "1"      , "null" , "null" },//
0
                        { "null" , "null" , "null" , "null" , "acc " , "null" , "null" , "null" },// 1
                        { "null" , "null" , "S4"  , "S10"  , "null" , "null" , "6"    , "null" },// 2
                        { "null" , "null" , "S5"  , "S11"  , "null" , "null" , "7"    , "null" },// 3
                        { "null" , "null" , "S4"  , "S10"  , "null" , "null" , "8"    , "null" },// 4
                        { "null" , "null" , "S5"  , "S11"  , "null" , "null" , "9"    , "null" },// 5
                        { "r1"   , "r1"   , "r1"   , "r1"   , "r1"   , "null" , "null" , "null" },//
6
                        { "r2"   , "r2"   , "r2"   , "r2"   , "r2"   , "null" , "null" , "null" },//
7
                        { "r3"   , "r3"   , "r3"   , "r3"   , "r3"   , "null" , "null" , "null" },//
8
                        { "r5"   , "r5"   , "r5"   , "r5"   , "r5"   , "null" , "null" , "null" },//
9
                        { "r4"   , "r4"   , "r4"   , "r4"   , "r4"   , "null" , "null" , "null" },//
10
                        { "r6"   , "r6"   , "r6"   , "r6"   , "r6"   , "null" , "null" , "null" },//
11
                        };
char L[200]="abcd#EAB";    ///列判断依据
int del[10]={0,2,2,2,1,2,1};
char head[20]={'S','E','E','A','A','B','B'};
stack<int>con;    ///状态栈
stack<char>cmp;    ///符号栈
char cod[300]="0";///初始状态栈对应输出数组
int cindex = 0;
char sti[300]="#";///初始符号栈对应输出数组
int sindex = 0;
int findL(char b)///对应列寻找
{
```

```

    for(int i = 0; i <= 7; i++)
    {
        if(b==L[i])
        {
            return i;
        }
    }
    return -1;
}

void error(int x, int y)        ///报错输出
{
    printf("第%d 行%c 列为空!",x,L[y]);
}

int calculate(int l, char s[])
{
    int num = 0;
    for(int i = 1; i < l; i++)
    {
        num = num*10+(s[i]-'0');
    }
    return num;
}

void analyze(char str[],int len)///分析主体过程
{
    int cnt = 1;
    printf("步骤      状态栈      符号栈      输入串      ACTION      GOTO\n");
    int LR = 0;
    while(LR<=len)
    {
        printf("(%d)      %-10s%-10s",cnt,cod,sti);///步骤，状态栈，符号栈输出
        cnt++;
        for(int i = LR; i < len; i++)///输入串输出
        {
            printf("%c",str[i]);
        }
        for(int i = len-LR; i<10;i++)printf(" ");

        int x = con.top();///状态栈栈顶
        int y = findL(str[LR]);///待判断串串首

        if(strcmp(LR0[x][y],"null")!=0)
        {
            int l = strlen(LR0[x][y]);///当前 Ri 或 Si 的长度

```

```

if(LR0[x][y][0]=='a')//acc
{
    printf("acc      \n");//ACTION 与 GOTO
    return ;
}
else if(LR0[x][y][0]=='S')//Si
{
    printf("%-10s \n",LR0[x][y]);//ACTION 与 GOTO
    int t = calculate(l,LR0[x][y]);//整数
    con.push(t);
    sindex++;
    sti[sindex] = str[LR];
    cmp.push(str[LR]);
    if(t<10)
    {
        cindex++;
        cod[cindex] = LR0[x][y][1];
    }
    else
    {
        int k = 1;
        cindex++;
        cod[cindex] = '(';
        while(k<l)
        {
            cindex++;
            cod[cindex] = LR0[x][y][k];
            k++;
        }
        cindex++;
        cod[cindex] = ')';
    }
    LR++;
}
else if(LR0[x][y][0]=='r')//ri,退栈, ACTION 和 GOTO
{
    printf("%-10s",LR0[x][y]);
    int t = calculate(l,LR0[x][y]);
    int g = del[t];
    while(g--)
    {
        con.pop();
        cmp.pop();
    }
}

```

```

        sti[sindex] = '\0';
        sindex--;
    }
    g = del[t];
    while(g>0)
    {
        if(cod[cindex]==')')
        {
            cod[cindex]='\0';
            cindex--;
            for(;;)
            {
                if(cod[cindex]=='(')
                {
                    cod[cindex]='\0';
                    cindex--;
                    break;
                }else
                {
                    cod[cindex]='\0';
                    cindex--;
                }
            }
            g--;
        }else
        {
            cod[cindex] = '\0';
            cindex--;
            g--;
        }
    }
    }///
    cmp.push(head[t]);
    sindex++;
    sti[sindex] = head[t];
    x = con.top();
    y = findL(cmp.top());
    t = LR0[x][y][0]-'0';
    con.push(t);
    cindex++;
    cod[cindex] = LR0[x][y][0];
    printf("%-10d\n",t);
}else
{
    int t = LR0[x][y][0]-'0';

```


输出:

步骤	状态栈	符号栈	输入串	ACTION	GOTO
(1)	0	#	bccd#	S3	
(2)	03	#b	ccd#	S5	
(3)	035	#bc	cd#	S5	
(4)	0355	#bcc	d#	S11	
(5)	0355(11)	#bccd	#	r6	9
(6)	03559	#bccB	#	r5	9
(7)	0359	#bcB	#	r5	7
(8)	037	#bB	#	r2	1
(9)	01	#E	#	acc	

输入: abc

输出:

步骤	状态栈	符号栈	输入串	ACTION	GOTO
(1)	0	#	abc	S2	
(2)	02	#a	bc	第 2 行 b 列为空!	

输入: abc#

输出:

步骤	状态栈	符号栈	输入串	ACTION	GOTO
(1)	0	#	abc#	S2	
(2)	02	#a	bc#	第 2 行 b 列为空!	

输入: E#

输出:

步骤	状态栈	符号栈	输入串	ACTION	GOTO
(1)	0	#	E#		1
(2)	01	#E	#	acc	

输入: cabc#

输出:

步骤	状态栈	符号栈	输入串	ACTION	GOTO
(1)	0	#	cabc#	第 0 行 c 列为空!	

输入: bccdd#

输出:

步骤	状态栈	符号栈	输入串	ACTION	GOTO
(1)	0	#	bccdd#	S3	
(2)	03	#b	ccdd#	S5	
(3)	035	#bc	cdd#	S5	
(4)	0355	#bcc	dd#	S11	
(5)	0355(11)	#bccd	d#	r6	9
(6)	03559	#bccB	d#	r5	9
(7)	0359	#bcB	d#	r5	7
(8)	037	#bB	d#	r2	1
(9)	01	#E	d#	第 1 行 d 列为空! */	